AI ASSISTED CODING

ASSIGNMENT-9.2

Name: Shashikanth G

HT NO: 2403A51245

Batch No.: 11

TASK – 1

Task: Use AI to add Google-style docstrings to all functions in a given Python script.

• Instructions:

o Prompt AI to generate docstrings without providing any input-output examples.

o Ensure each docstring includes:

▪ Function description

▪ Parameters with type hints

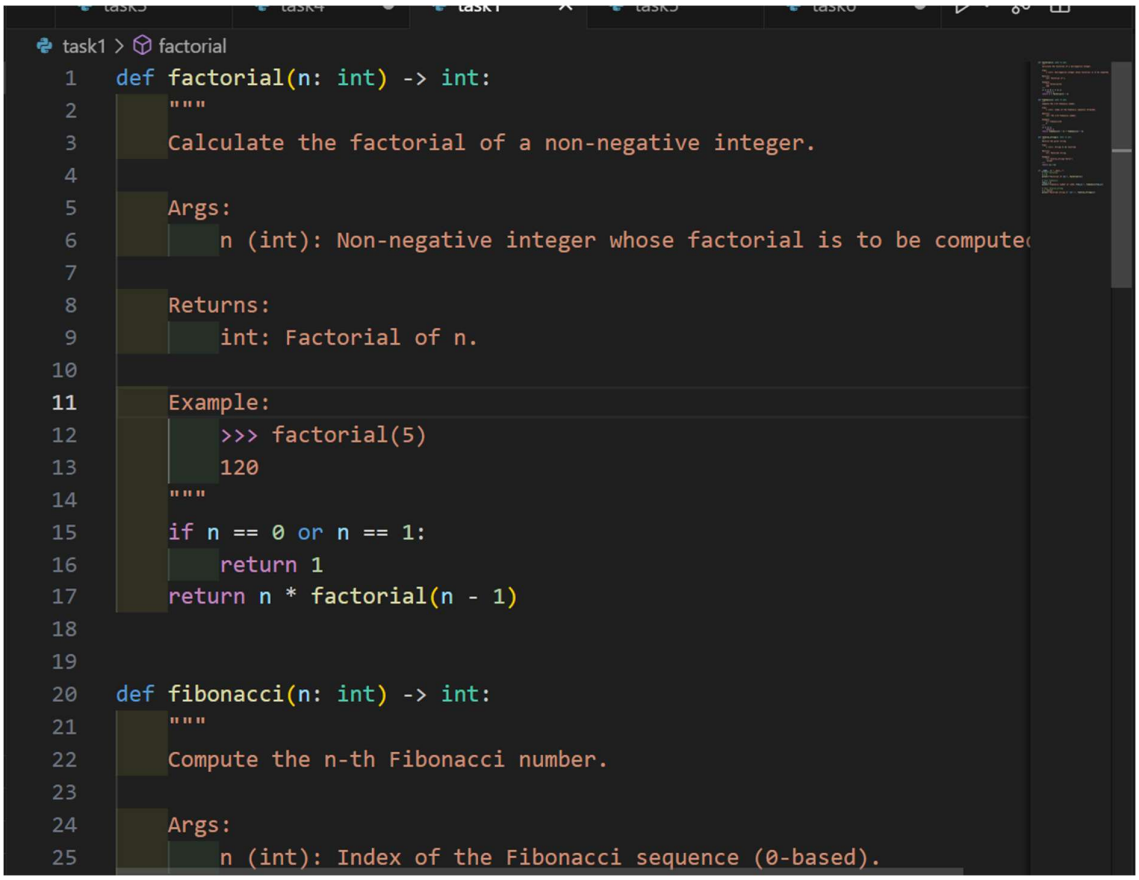
▪ Return values with type hints

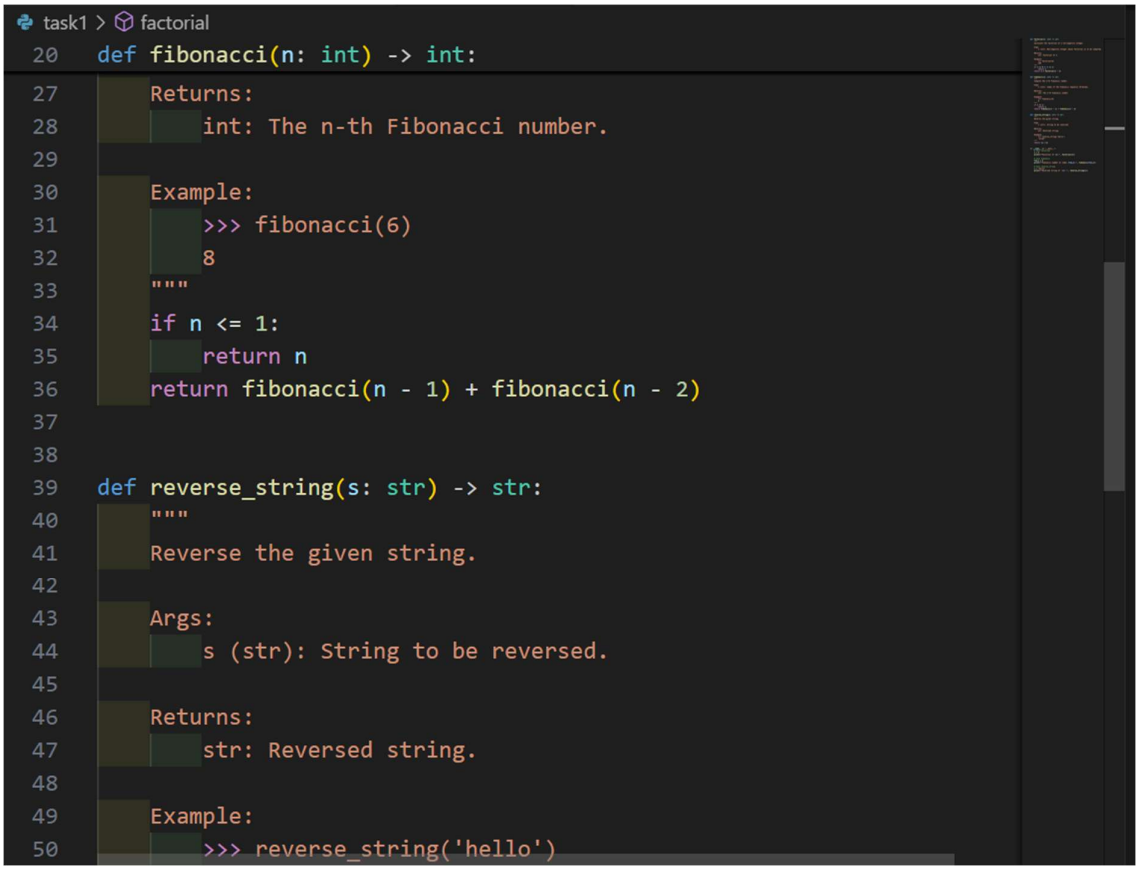
▪ Example usage

o Review the generated docstrings for accuracy and formatting.

Expected Output 1:

o A Python script with all functions documented using correctly formatted Google-style docstrings.


task1 > factorial
def factorial(n: int) -> int:
"""
Calculate the factorial of a non-negative integer.
Args:
n (int): Non-negative integer whose factorial is to be computed
Returns:
int: Factorial of n.
Example:
>>> factorial(5)
120
"""
if n == 0 or n == 1:
return 1
return n * factorial(n - 1)
def fibonacci(n: int) -> int:
"""
Compute the n-th Fibonacci number.
Args:
n (int): Index of the Fibonacci sequence (0-based).


task1 > factorial
20 def fibonacci(n: int) -> int:
27 Returns:
28 int: The n-th Fibonacci number.
29
30 Example:
31 >>> fibonacci(6)
32 8
33 """
34 if n <= 1:
35 return n
36 return fibonacci(n - 1) + fibonacci(n - 2)
37
38
39 def reverse_string(s: str) -> str:
40 """
41 Reverse the given string.
42
43 Args:
44 s (str): String to be reversed.
45
46 Returns:
47 str: Reversed string.
48
49 Example:
50 >>> reverse_string('hello')

```python
task1 > factorial
def reverse_string(s: str) -> str:
    Returns:
        str: Reversed string.

    Example:
        >>> reverse_string('hello')
        'olleh'
    """
    return s[::-1]


if __name__ == "__main__":
    # Test factorial
    n = 5
    print(f"Factorial of {n}:", factorial(n))

    # Test fibonacci
    fib_n = 6
    print(f"Fibonacci number at index {fib_n}:", fibonacci(fib_n))

    # Test reverse_string
    s = "hello"
    print(f"Reversed string of '{s}':", reverse_string(s))
```

OUTPUT:

```
PS C:\AI> python -u "c:\AI\task1"
Factorial of 5: 120
Fibonacci number at index 6: 8
Reversed string of 'hello': olleh
PS C:\AI>
```

TASK – 2

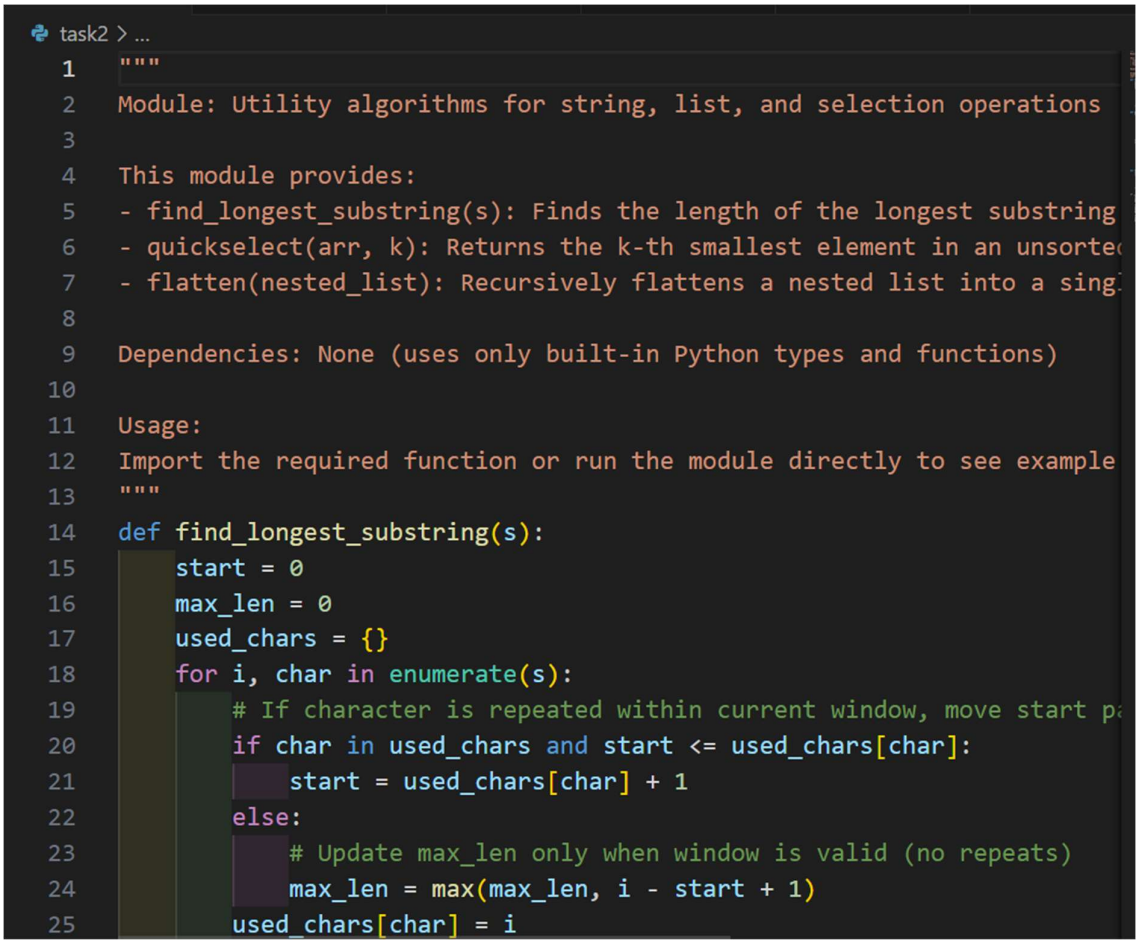
Task: Use AI to add meaningful inline comments to a Python program explaining only complex logic parts.

• Instructions:

o Provide a Python script without comments to the AI.

o Instruct AI to skip obvious syntax explanations and focus only on tricky or non-intuitive code sections.
o Verify that comments improve code readability and maintainability.

Expected Output 2:
o Python code with concise, context-aware inline comments for complex logic blocks.



```
task2 > ...
"""
Module: Utility algorithms for string, list, and selection operations

This module provides:
- find_longest_substring(s): Finds the length of the longest substring
- quickselect(arr, k): Returns the k-th smallest element in an unsorte
- flatten(nested_list): Recursively flattens a nested list into a sing

Dependencies: None (uses only built-in Python types and functions)

Usage:
Import the required function or run the module directly to see example
"""
def find_longest_substring(s):
    start = 0
    max_len = 0
    used_chars = {}
    for i, char in enumerate(s):
        # If character is repeated within current window, move start p
        if char in used_chars and start <= used_chars[char]:
            start = used_chars[char] + 1
        else:
            # Update max_len only when window is valid (no repeats)
            max_len = max(max_len, i - start + 1)
        used_chars[char] = i
```

task2 > ...

```
14  def find_longest_substring(s):
25          used_chars[char] = i
26      return max_len
27
28
29  def quickselect(arr, k):
30      def partition(left, right):
31          pivot = arr[right]
32          i = left
33          for j in range(left, right):
34              # Place elements <= pivot to the left
35              if arr[j] <= pivot:
36                  arr[i], arr[j] = arr[j], arr[i]
37                  i += 1
38          # Final pivot placement
39          arr[i], arr[right] = arr[right], arr[i]
40          return i
41
42      left, right = 0, len(arr) - 1
43      while True:
44          idx = partition(left, right)
45          # Found kth smallest element
46          if idx == k:
47              return arr[idx]
48          elif idx < k:
```

task2 > ...

```
29  def quickselect(arr, k):
46          if idx == k:
47              return arr[idx]
48          elif idx < k:
49              # Search right partition
50              left = idx + 1
51          else:
52              # Search left partition
53              right = idx - 1
54
55
56  def flatten(nested_list):
57      result = []
58      for item in nested_list:
59          # Recursively flatten if item is a list
60          if isinstance(item, list):
61              result.extend(flatten(item))
62          else:
63              result.append(item)
64      return result
65
66
67  if __name__ == "__main__":
68      # Test find_longest_substring
69      s = "abcabcbb"
```

task2 > ...

```
56  def flatten(nested_list):
58      for item in nested_list:
59          # Recursively flatten if item is a list
60          if isinstance(item, list):
61              result.extend(flatten(item))
62          else:
63              result.append(item)
64      return result
65
66
67  if __name__ == "__main__":
68      # Test find_longest_substring
69      s = "abcabcbb"
70      print("Longest substring without repeating characters in '{}':".format(s), find_longest_substring(s))
71
72      # Test quickselect
73      arr = [3, 2, 1, 5, 4]
74      k = 2
75      print("{}th smallest element in {}:".format(k + 1, arr), quickselect(arr.copy(), k))
76
77      # Test flatten
78      nested = [1, [2, [3, 4], 5], 6]
79      print("Flattened list:", flatten(nested))
80
```

OUTPUT:

```
PS C:\AI> python -u "c:\AI\task2"
Longest substring without repeating characters in 'abcabcbb': 3
3th smallest element in [3, 2, 1, 5, 4]: 3
Flattened list: [1, 2, 3, 4, 5, 6]
PS C:\AI>
```

TASK – 3

Task: Use AI to create a module-level docstring summarizing the purpose, dependencies, and main functions/classes of a Python file.

• Instructions:
o Supply the entire Python file to AI.
o Instruct AI to write a single multi-line docstring at the top of the file.
o Ensure the docstring clearly describes functionality and usage without rewriting the entire code.

Expected Output 3:
o A complete, clear, and concise module-level docstring at the beginning of the file.

task3 > ...

```python
"""
Module: Utility functions and class for string, interval, and counting operations

This module provides:
- is_palindrome(s): Checks if a string is a palindrome, ignoring case and non-alphanumeric characters.
- merge_intervals(intervals): Merges overlapping intervals in a list of [start, end] pairs.
- Counter class: Simple counter for tracking occurrences of items.

Dependencies: None (uses only built-in Python types and functions)

Usage:
Import the required function or class, or instantiate Counter to count items in your code.
"""
def is_palindrome(s):
    s = ''.join(c.lower() for c in s if c.isalnum())
    return s == s[::-1]


def merge_intervals(intervals):
    intervals.sort(key=lambda x: x[0])
    merged = []
    for interval in intervals:
        if not merged or merged[-1][1] < interval[0]:
            merged.append(interval)
        else:
```

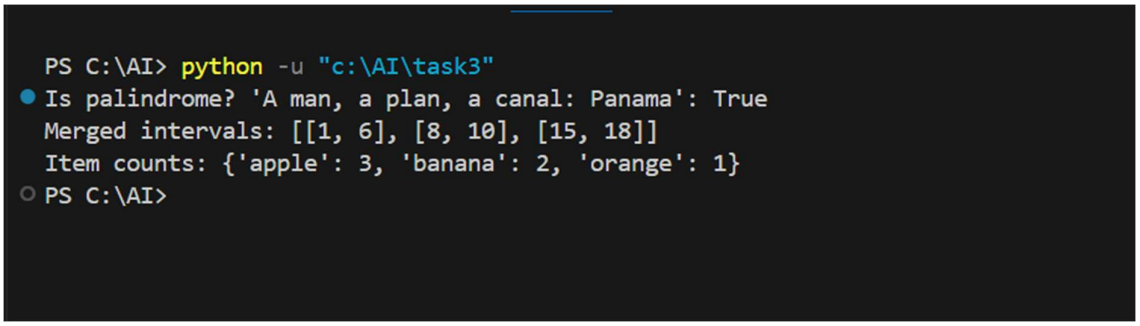
task3 > ...

```python
def merge_intervals(intervals):
    merged = []
    for interval in intervals:
        if not merged or merged[-1][1] < interval[0]:
            merged.append(interval)
        else:
            merged[-1][1] = max(merged[-1][1], interval[1])
    return merged


class Counter:
    def __init__(self):
        self.counts = {}

    def add(self, item):
        self.counts[item] = self.counts.get(item, 0) + 1

    def get(self, item):
        return self.counts.get(item, 0)


if __name__ == "__main__":
    # Test is_palindrome
    s = "A man, a plan, a canal: Panama"
    print("Is palindrome? '{}':".format(s), is_palindrome(s))
```

task3 > ...

```python
class Counter:
        return self.counts.get(item, 0)


if __name__ == "__main__":
    # Test is_palindrome
    s = "A man, a plan, a canal: Panama"
    print("Is palindrome? '{}':".format(s), is_palindrome(s))

    # Test merge_intervals
    intervals = [[1, 3], [2, 6], [8, 10], [15, 18]]
    print("Merged intervals:", merge_intervals([i[:] for i in intervals]))

    # Test Counter class
    counter = Counter()
    items = ['apple', 'banana', 'apple', 'orange', 'banana', 'apple']
    for item in items:
        counter.add(item)
    print("Item counts:", {item: counter.get(item) for item in set(items)})
```

OUTPUT:



TASK – 4

Task: Use AI to transform existing inline comments into structured function docstrings following Google style.

• Instructions:

o Provide AI with Python code containing inline comments.

o Ask AI to move relevant details from comments into function docstrings.

o Verify that the new docstrings keep the meaning intact while improving structure.

Expected Output 4:

o Python code with comments replaced by clear, standardized docstrings.

task4 > greet

```python
def sum_list(numbers):
    """
    Calculate the sum of all elements in a list.

    Args:
        numbers (list of int): List of integers to sum.

    Returns:
        int: The sum of all elements in the list.
    """
    total = 0
    for num in numbers:
        total += num
    return total


def is_even(n):
    """
    Check if a number is even.

    Args:
        n (int): Number to check.

    Returns:
        bool: True if n is even, False otherwise.
    """
```

task4 > greet

```python
def is_even(n):
    return n % 2 == 0


def greet(name):
    """
    Print a greeting message to the user.

    Args:
        name (str): Name of the person to greet.

    Returns:
        None
    """
    print("Hello, {}!".format(name))


    if __name__ == "__main__":
        # Test sum_list
        numbers = [1, 2, 3, 4, 5]
        print("Sum of list:", sum_list(numbers))

        # Test is_even
        n = 4
        print(f"Is {n} even?", is_even(n))
```

```
task4 > greet
30  def greet(name):
36
37      Returns:
38          None
39      """
40      print("Hello, {}!".format(name))
41
42
43      if __name__ == "__main__":
44          # Test sum_list
45          numbers = [1, 2, 3, 4, 5]
46          print("Sum of list:", sum_list(numbers))
47
48          # Test is_even
49          n = 4
50          print(f"Is {n} even?", is_even(n))
51
52          # Test greet
53          greet("Alice")
```

OUTPUT:

```
PS C:\AI> python -u "c:\AI\task4"
Sum of list: 15
Is 4 even? True
Hello, Alice!
PS C:\AI>
```

TASK – 5

Task: Use AI to identify and correct inaccuracies in existing docstrings.

• Instructions:

o Provide Python code with outdated or incorrect docstrings.

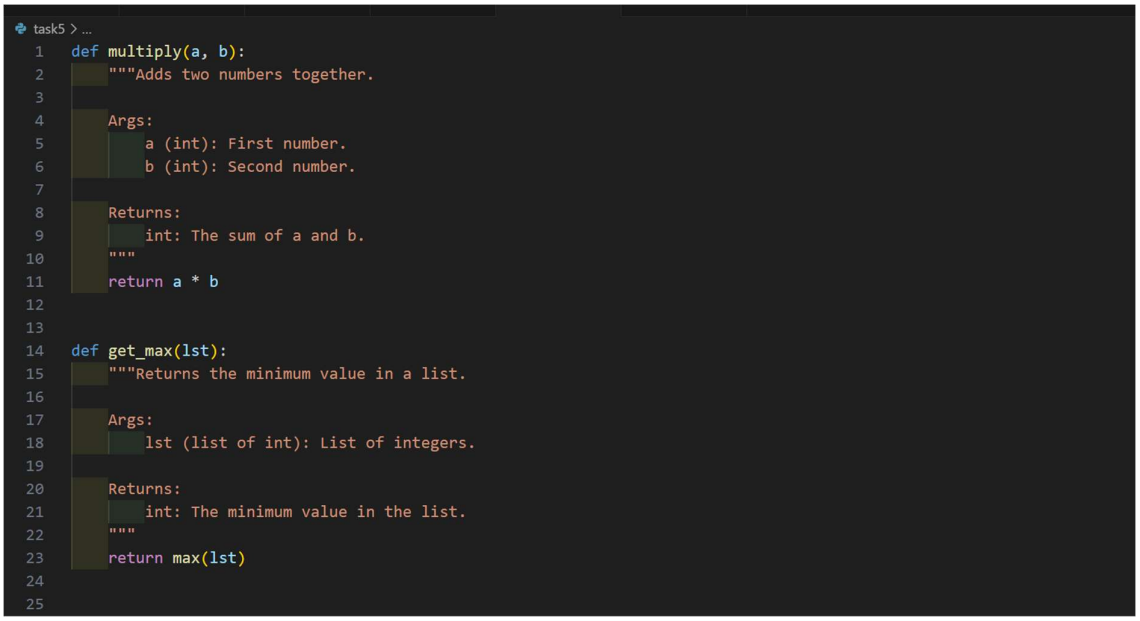
o Instruct AI to rewrite each docstring to match the current

code behavior.

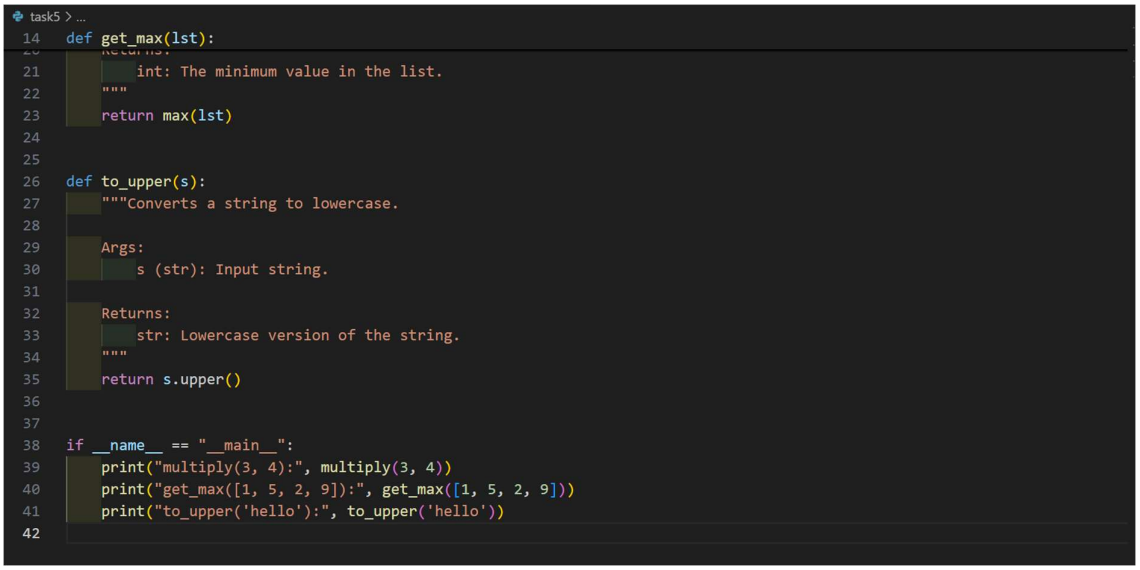
o Ensure corrections follow Google-style formatting.

Expected Output 5:

o Python file with updated, accurate, and standardized docstrings.

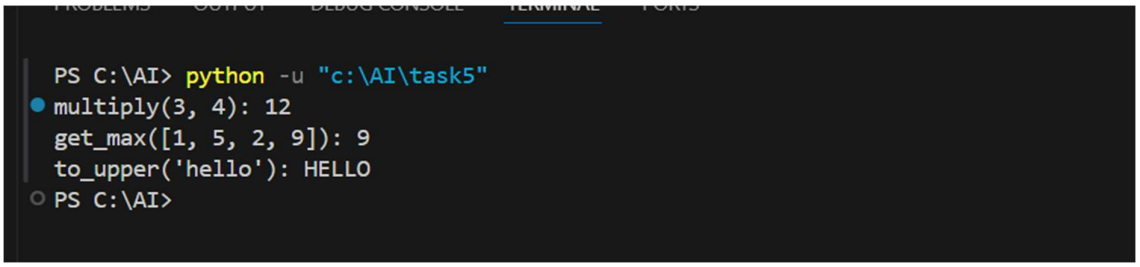


```
task5 > ...
def multiply(a, b):
    """Adds two numbers together.

    Args:
        a (int): First number.
        b (int): Second number.

    Returns:
        int: The sum of a and b.
    """
    return a * b


def get_max(lst):
    """Returns the minimum value in a list.

    Args:
        lst (list of int): List of integers.

    Returns:
        int: The minimum value in the list.
    """
    return max(lst)
```



```
task5 > ...
def get_max(lst):
        int: The minimum value in the list.
    """
    return max(lst)


def to_upper(s):
    """Converts a string to lowercase.

    Args:
        s (str): Input string.

    Returns:
        str: Lowercase version of the string.
    """
    return s.upper()


if __name__ == "__main__":
    print("multiply(3, 4):", multiply(3, 4))
    print("get_max([1, 5, 2, 9]):", get_max([1, 5, 2, 9]))
    print("to_upper('hello'):", to_upper('hello'))
```

OUTPUT:


PS C:\AI> python -u "c:\AI\task5"
multiply(3, 4): 12
get_max([1, 5, 2, 9]): 9
to_upper('hello'): HELLO
PS C:\AI>